

Project 2: Purple Team Operations – Active Directory Threat Simulation & Remediation

1. Executive Summary

Active Directory (AD) is the primary identity provider for the enterprise, making it a high-priority target for advanced persistent threats (APTs). Because default Windows security auditing often obscures critical security events within massive volumes of benign background noise, organizations face severe visibility gaps.

The objective of this project was to simulate two critical identity-based attacks—Kerberoasting and LSASS Credential Access—using Atomic Red Team, analyze the resulting telemetry, and engineer high-fidelity custom SIEM rules in Wazuh. This project demonstrates the ability to close native OS blind spots using Microsoft Sysmon, bypass strict SIEM parsing limitations, and translate raw telemetry into actionable, low-noise alerts for a Security Operations Center (SOC).

2. Target Role & Technical Skills

Target Role: Cybersecurity Analyst / Detection Engineer

Demonstrated Technical Skills:

- **SIEM & Log Management:** Wazuh (v4.14.1), OSSEC Engine, Microsoft Sysmon, Wazuh Agent log collection, JSON Log Parsing.
- **Threat Detection & Engineering:** Custom Detection Rules (XML, PCRE2 Regex), Sigma Rule translation, MITRE ATT&CK Framework.
- **Offensive Security & Simulation:** Atomic Red Team, Kerberoasting (Rubeus), LSASS Process Access (`comsvcs.dll`, `rdrleakdiag.exe`).
- **Infrastructure & Systems:** Active Directory (Windows Server 2019 Domain Controller), Windows 11 Enterprise, VirtualBox (isolated Host-Only Networking).
- **Infrastructure Troubleshooting:** Diagnosed and resolved complex JSON parsing failures, pipeline ingestion drops, and Windows Group Policy overrides.

3. Lab Architecture

The environment was constructed within Oracle VirtualBox on an isolated Host-Only network (`192.168.56.x`) to safely simulate domain compromise and lateral movement telemetry.

- **SIEM Manager:** Wazuh Manager (v4.14.1) running on Ubuntu Linux (IP: `192.168.56.105`). Central engine for log ingestion, decoding, rule evaluation, and indexing.
- **Domain Controller (DC01):** Windows Server 2019 (IP: `192.168.56.110`) managing the `lab.local` domain. Enforces Kerberos authentication policies and advanced audit logging.
- **Workstation / Attack Host (Win11-Client):** Windows 11 Enterprise (IP: `192.168.56.101`). Domain-joined workstation equipped with the Wazuh Agent, Microsoft Sysmon, and the Atomic Red Team execution framework.

4. Threat Emulation & Detection Engineering

Scenario A: Kerberoasting (T1558.003)

Business Risk

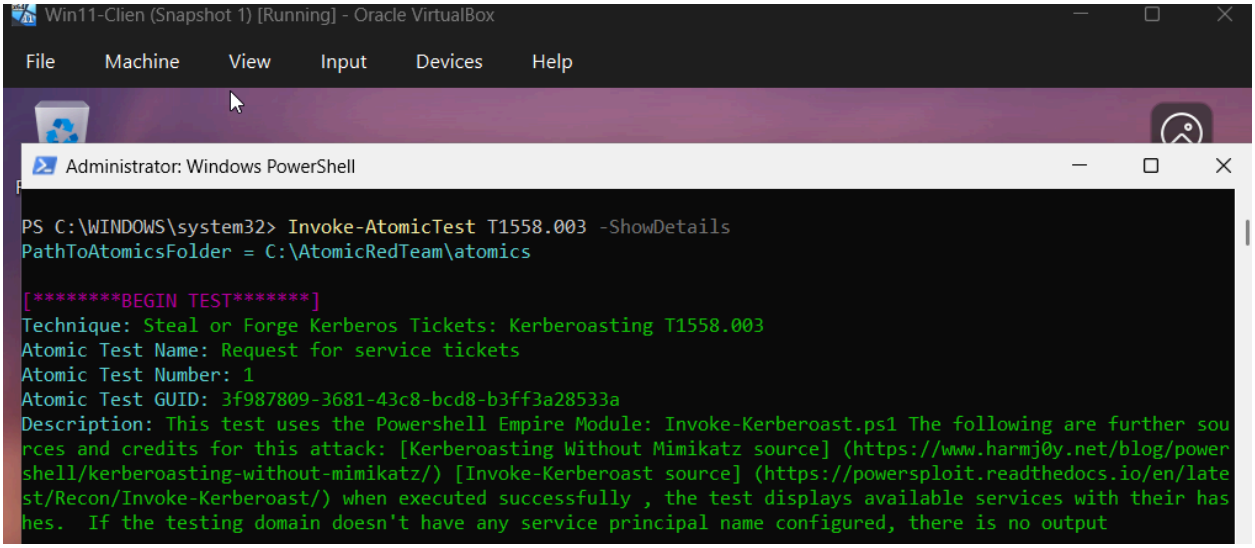
Any authenticated domain account can request a Ticket Granting Service (TGS) ticket for service accounts with registered Service Principal Names (SPNs). Attackers extract these tickets from memory and crack them offline to recover cleartext passwords, exposing backend infrastructure.

Note: While an RC4 ticket request (0x17) is a strong signal for investigation, it can also result from legitimate legacy systems (e.g., Linux-integrated accounts) and does not by itself definitively prove Kerberoasting.

The Simulation

Atomic Red Team (`Rubeus`) was utilized to request an RC4-encrypted (`0x17`) ticket for a target SPN (`svc_sql`) from the Windows 11 client.

```
klist purge
Invoke-AtomicTest T1558.003
```



Evidence: Kerberos Telemetry (Event 4769)

On `DC01`, the Kerberos ticket request was logged under Event ID 4769. The critical forensic indicators were `Ticket Options: 0x40810000`, `Ticket Encryption Type: 0x17` (legacy RC4-HMAC), and `Service Name: svc_sql`.

: Atomic Red Team / Rubeus output displaying the extracted `$krb5tgs$23$*svc_sql...` RC4 ticket hash.

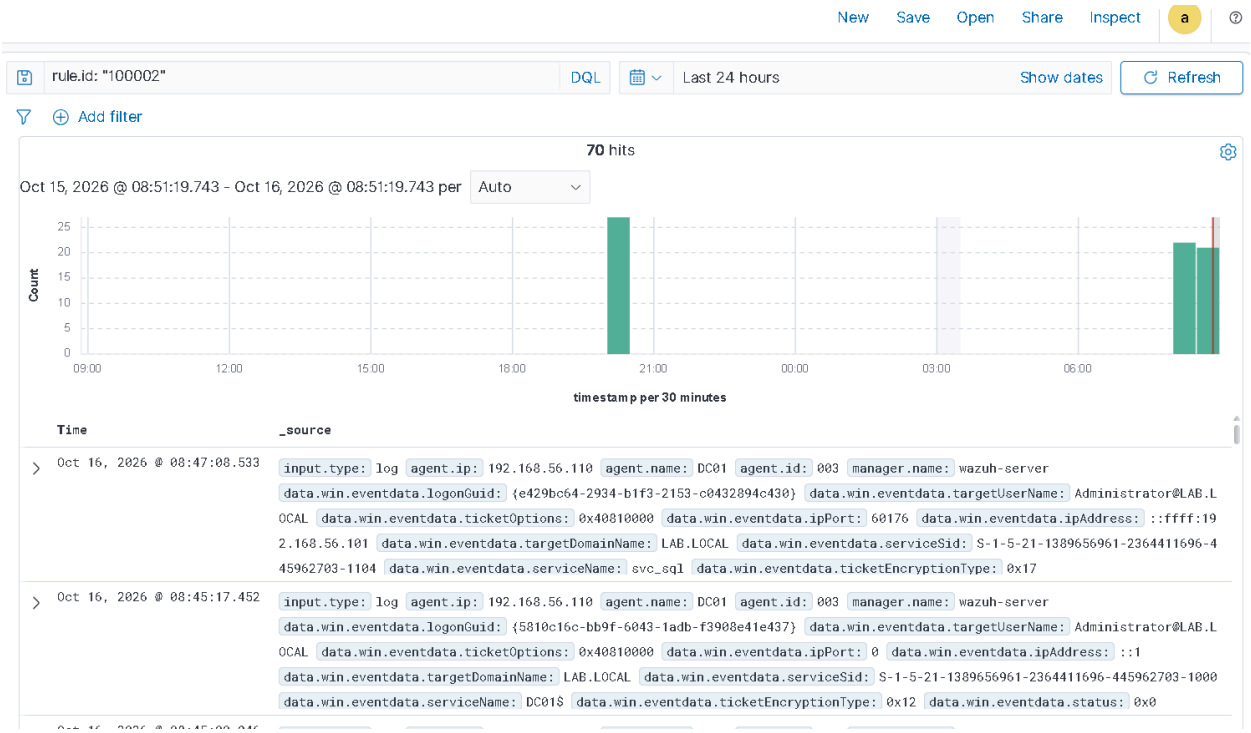
Rule: Kerberoasting Detection (ID: 100002)

Native Event ID 4769 generates extreme background noise. Wazuh evaluates these events under a baseline rule chain starting with Rule `60103`. To elevate true threats, this custom rule hooks directly into parent rule `60103`, explicitly filters for the `0x17` encryption type, and uses a case-insensitive PCRE2 regex to isolate the targeted user service account.

Custom Wazuh Rule (local_rules.xml):

```
<group name="windows,active_directory,kerberoasting,">
  <rule id="100002" level="12">
    <if_sid>60103</if_sid>
    <field name="win.system.eventID">^4769$</field>
    <field name="win.eventdata.ticketEncryptionType">^0x17$</field>
    <field name="win.eventdata.serviceName" type="pcre2">(?i).*svc_sql.*</field>
    <description>Active Directory: Potential Kerberoasting - RC4 ticket requested for svc
_sql</description>
    <mitre>
      <id>T1558.003</id>
    </mitre>
    <group>attack,kerberoasting,</group>
  </rule>
</group>
```

Wazuh Level 12 Alert successfully isolating the malicious `0x17` ticket request while filtering out benign domain traffic.



Scenario B: LSASS Credential Access (T1003.001)

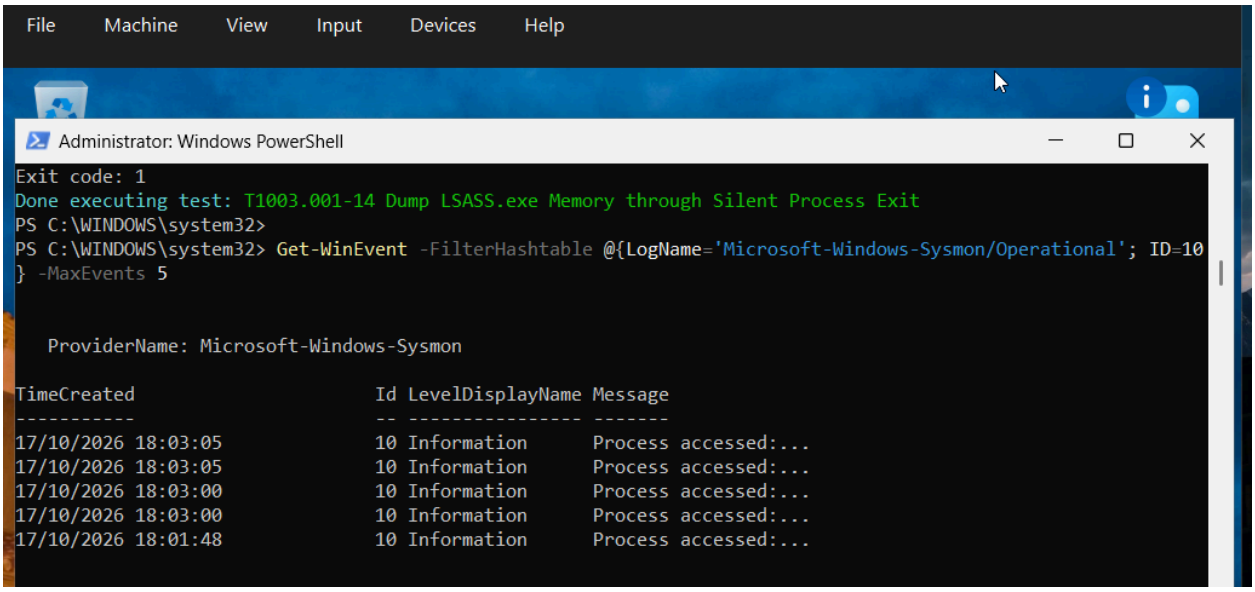
Business Risk

The Local Security Authority Subsystem Service (`lsass.exe`) caches active credentials, Kerberos tickets, and NTLM password hashes. Accessing LSASS allows attackers to escalate privileges locally and move laterally across enterprise systems.

The Simulation

Atomic Red Team was used to execute credential access modules, invoking native Living-off-the-Land Binaries (LOLBins) like `comsvcs.dll` and `rdrleakdiag.exe` directly on the client to target LSASS.

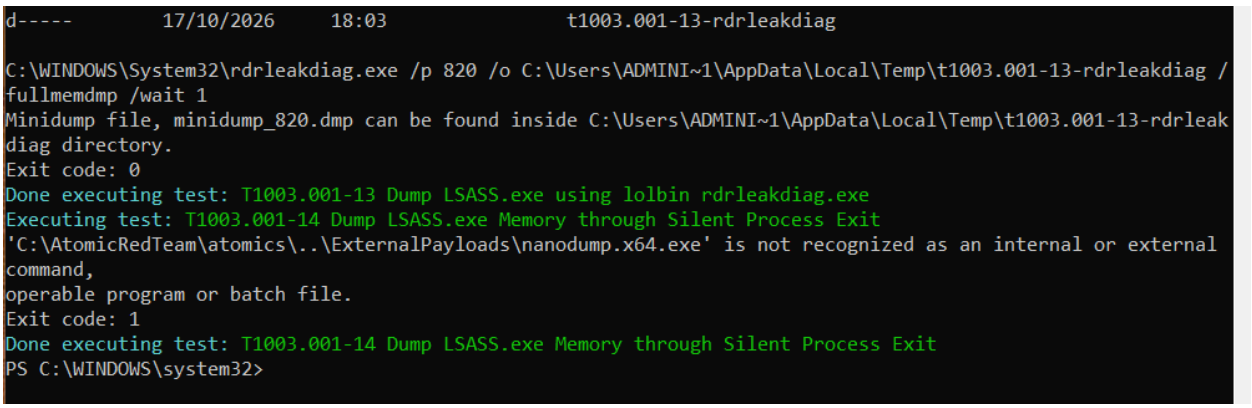
```
Invoke-AtomicTest T1003.001
```



Evidence: Sysmon Kernel-Level Process Access (Event 10)

Native Windows event logs do not deeply monitor process-to-process access. I deployed Microsoft Sysmon to explicitly target `lsass.exe`. Sysmon captured Event ID 10 with `GrantedAccess: 0x1000`, indicating limited process-information access. This event alone does not prove LSASS memory was read; look for the `0x0010` memory-read bit in `GrantedAccess` (often appearing in combined masks like `0x1010`) and correlate with other evidence.

Execution of LSASS memory extraction techniques via Atomic Red Team.



Rule: LSASS Process Access Detection (ID: 100003)

I configured the Windows 11 Wazuh agent (`ossec.conf`) to actively ingest the `Microsoft-Windows-Sysmon/Operational` event channel. The manager rule checks for Sysmon Event 10 where the target process image matches `lsass.exe`.

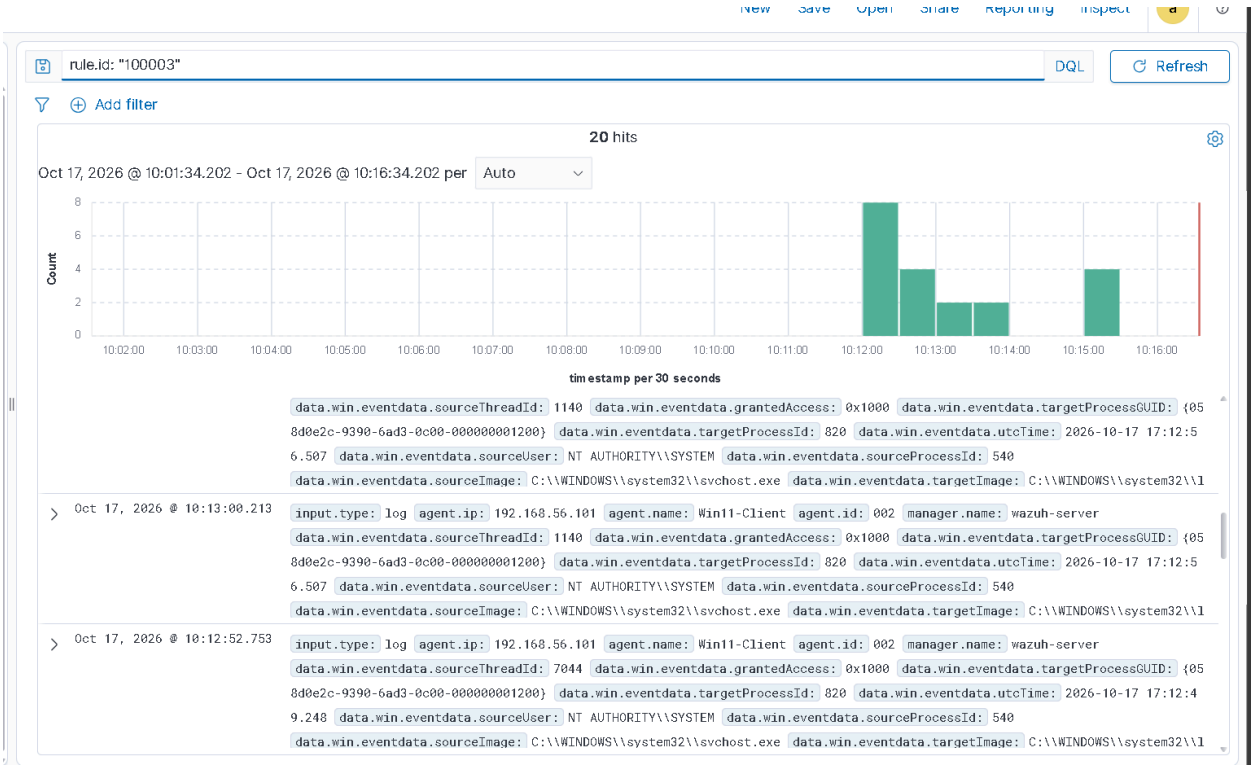
Custom Wazuh Rule (local_rules.xml):

```
<!-- General LSASS Access (Low noise tracker) -->
  <rule id="100003" level="5">
    <if_group>windows</if_group>
    <field name="win.system.eventID">^10$</field>
    <match>lsass.exe</match>
    <description>Sysmon: Process accessed LSASS memory (Informational/Low Level).</descri
ption>
  </rule>

  <!-- High-Fidelity LSASS Memory Read (Credential Dumping) -->
  <rule id="100004" level="12">
    <if_sid>100003</if_sid>
    <field name="win.eventdata.grantedAccess" type="pcre2">(?i)^0x(1010|1410|1fffff)$</fi
eld>
    <description>Sysmon: High-fidelity LSASS memory read access detected (Potential Crede
ntial Dumping).</description>
    <mitre>
      <id>T1003.001</id>
    </mitre>
  </rule>
```

By splitting the detection, Rule 100003 acts as a broad tracker for all LSASS access, while Rule 100004 provides a high-fidelity alert specifically for the memory-read access masks (0x1010, 0x1410, 0x1fffff) observed during credential dumping."

Wazuh Level 12 Alert detecting potential LSASS process access via Sysmon telemetry.



5. Vendor-Neutral Sigma Rule Translation

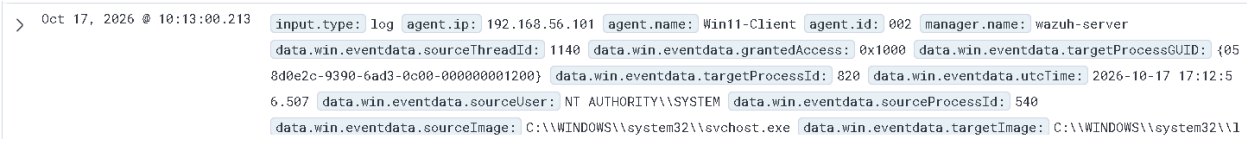
To ensure portable detection across enterprise platforms (Splunk, Microsoft Sentinel, Elastic), I translated the Kerberoasting logic into a vendor-neutral Sigma rule, leveraging named filter groups for precise exclusion:

```
title: Potential Kerberoasting Active Directory Attack
id: 3c267c42-8812-4f35-99d8-912809b0b14f
status: experimental
description: Detects Kerberos service ticket requests (TGS) utilizing weak RC4 encryption
(0x17) against user service accounts. Note that 0x17 alone does not guarantee malicious a
ctivity, as legacy systems may legitimately use RC4-HMAC.
author: Ellzy
date: 2026-10-17
references:
    - <https://attack.mitre.org/techniques/T1558/003/>
tags:
    - attack.credential_access
    - attack.t1558.003
logsource:
    product: windows
    service: security
detection:
    selection:
        EventID: 4769
```

```
        TicketEncryptionType: '0x17'
    filter_machine_accounts:
        ServiceName|endswith: '$'
    filter_krbtgt:
        ServiceName: 'krbtgt'
    condition: selection and not filter_machine_accounts and not filter_krbtgt
fields:
    - TargetUserName
    - ServiceName
    - TicketOptions
    - IpAddress
falsepositives:
    - Legacy applications or non-Windows operating systems lacking AES support
level: high
```

6. Incident Response & SOC Triage Playbook

When Phase 2 detections (100002 or 100004) trigger in an unhardened or partially hardened production environment, SOC analysts should follow this structured triage workflow:



Playbook A: Kerberoasting Alert (Rule 100002 — Level 12)

1. **Validate Source & Identity:** Inspect `data.win.eventdata.ipAddress` and `data.win.eventdata.targetUserName` in the Wazuh alert to identify which workstation and domain user requested the `0x17` TGS ticket.
2. **Assess Target SPN Risk:** Query Active Directory for the targeted `data.win.eventdata.serviceName` (e.g., `svc_sql`). Check administrative group memberships (`Domain Admins`, `Backup Operators`), password age (`pwdLastSet`), and supported encryption attributes (`msDS-SupportedEncryptionTypes`).
3. **Hunt for Burst Behavior:** Search Event ID 4679 across a 5-minute window from the same source IP. A single `0x17` request may be a legacy application; dozens of `0x17` requests across multiple SPNs indicate automated enumeration (`Rubeus` / `GetUserSPNs.py`).

4. **Correlate Post-Exploitation:** A ticket request alone does not prove the password was cracked. Hunt for subsequent Event ID 4624 (Logon Type 3 or 10) or Event ID 4672 (Special Privileges Assigned) for `svc_sql` originating from unusual internal hosts.
5. **Conditional Containment:** If anomalous logons are observed—or if the SPN uses a weak password—rotate the service account password twice (to invalidate existing tickets), isolate the source host, and migrate the account to a Group Managed Service Account (gMSA).

Playbook B: LSASS Process Access Alert (Rule 100004 — Level 12)

1. **Examine the Accessing Process:** Review `data.win.eventdata.sourceImage` and `data.win.eventdata.sourceProcessId` in the Sysmon Event ID 10 payload to identify the binary touching `lsass.exe`.
2. **Analyze the Access Mask:** Inspect `data.win.eventdata.grantedAccess`. Masks such as `0x1000` (`PROCESS_QUERY_LIMITED_INFORMATION`) indicate metadata queries, whereas masks containing the `0x0010` bit (`PROCESS_VM_READ`—such as `0x1010`, `0x1410`, or `0x1fffff`) indicate active memory reading capable of credential extraction.
3. **Correlate File Creation:** Pivot in Wazuh to Sysmon Event ID 11 (FileCreate) on the same host within 30 seconds of the alert to check if a `.dmp` or `.bin` file was written to `C:\Windows\Temp\` or `AppData\Local\Temp\`.
4. **Immediate Containment:** If `sourceImage` is an unapproved binary, scripting engine (`powershell.exe`, `rundll32.exe`), or LOLBin (`rdrleakdiag.exe`, `procdump.exe`), isolate the endpoint immediately, revoke active Kerberos TGTs for all users logged into that host, and initiate credential rotation.

7. False Positives & Detection Tuning

- **Kerberoasting (Rule 100002):**
  - **Common False Positives:** Legacy enterprise appliances (older NAS units, legacy printers) or non-Windows/Linux services joined to Active Directory that do not support AES encryption may legitimately negotiate `0x17` tickets.

    - **Tuning Strategy:** Audit and document legacy SPNs requiring RC4 while migration to AES is planned. Add specific `<field name="win.eventdata.serviceName" negate="yes">` exclusions in `local_rules.xml` for verified legacy service accounts rather than lowering the alert level globally.
- **LSASS Process Access (Rule 100004):**
    - **Common False Positives:** Legitimate Endpoint Detection and Response (EDR) sensors, antivirus scanners, and IT diagnostic tools (e.g., Microsoft Process Explorer, Task Manager) routinely access LSASS during system diagnostics.
    - **Tuning Strategy:** Tuning requires whitelisting verified binary paths, authentic digital signatures, and known binary hashes.

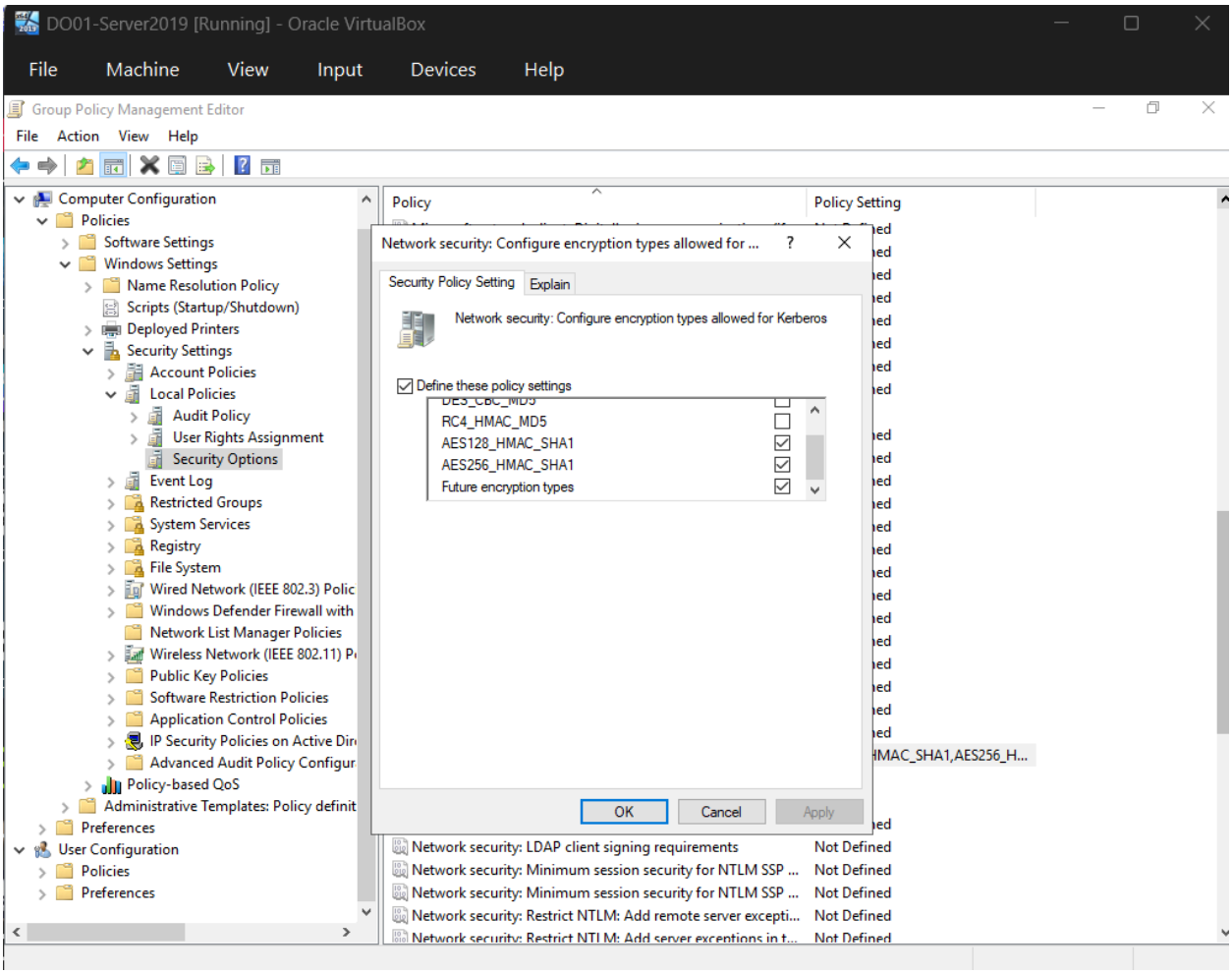
8. Response and Strategic Mitigations

Incident Containment

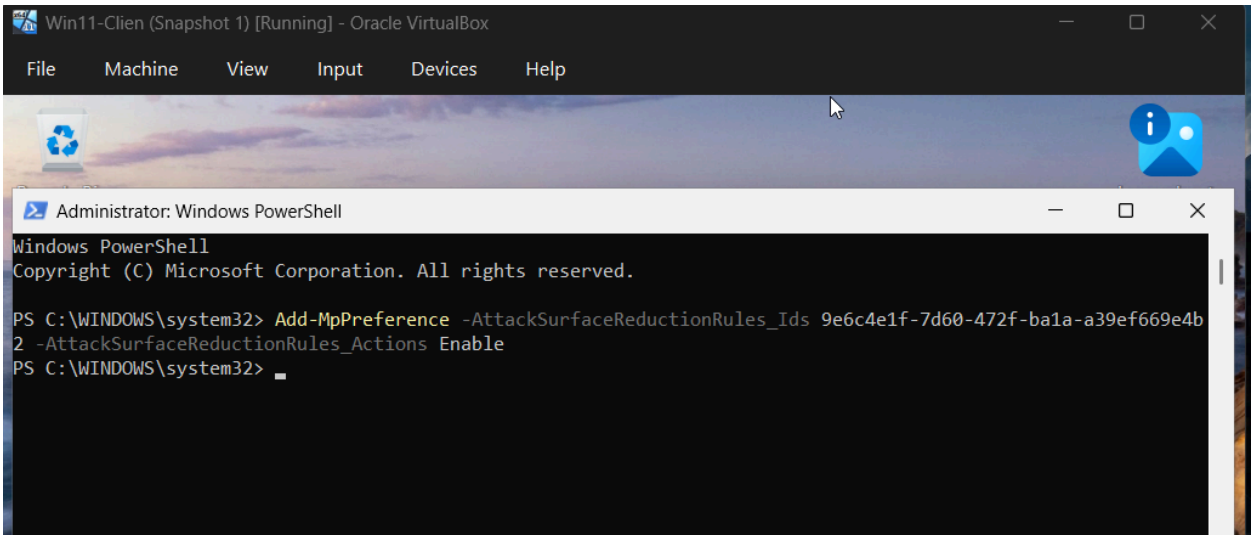
- If evidence confirms credential theft, revoke active sessions and force a password reset for the compromised accounts.
- Quarantine the source workstation to halt lateral movement.

Recommended Enterprise Mitigations (Architectural Defense)

- **Decommission RC4 Encryption:** Enforce AES-128 and AES-256 for Kerberos encryption domain-wide via Group Policy (`Network security: Configure encryption types allowed for Kerberos`). Completely disable DES and RC4 encryption types.



- **Service Account Hardening:** Transition standard service accounts to Group Managed Service Accounts (gMSA). gMSAs utilize 128-character, randomly generated passwords that automatically rotate, rendering offline dictionary cracking ineffective.
- **Enable Microsoft Defender Credential Guard:** Use virtualization-based security (VBS) on Windows endpoints to isolate credential secrets, preventing them from being accessed directly via the standard LSASS process. *Note: While it significantly reduces the attack surface, it does not block every form of credential access.*
- **Attack Surface Reduction (ASR):** Enable the Windows Defender ASR rule `Block credential stealing from the Windows local security authority subsystem` (`9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2`).



9. What This Demonstrated

- **Enterprise Infrastructure Deployment:** Built and configured a Windows Server 2019 Domain Controller to serve as a realistic target environment.
- **Adversarial Emulation:** Executed live, mapped MITRE ATT&CK techniques (T1558.003 Kerberoasting, T1003.001 OS Credential Dumping) using Atomic Red Team.
- **Endpoint Telemetry Generation:** Deployed Sysmon via GPO and ingested high-fidelity endpoint logs into Wazuh for centralized visibility.
- **Custom Detection Engineering:** Authored custom Wazuh SIEM rules using Regex/PCRE2 to detect specific attack artifacts, tuning out benign noise to create high-fidelity alerts.

10. Phase 3: Hardening & Remediation (Security Architecture)

To transition from a reactive detection posture (Phases 1 & 2) to a proactive defensive architecture, I engineered and deployed environmental hardening controls. The objective was to close the vulnerabilities exploited during the initial simulations utilizing Active Directory Group Policy (GPO) and Microsoft Defender protections.

Remediation 1: Defeating Kerberoasting (RC4 Disablement)

Technical Vulnerability

Kerberoasting fundamentally relies on the mathematical weakness of the legacy RC4-HMAC (`0x17`) encryption algorithm. When an authenticated user requests a Ticket Granting Service (TGS) ticket for an account with a registered Service Principal Name (SPN), the Domain Controller encrypts a portion of that ticket using the service account's NTLM password hash—if RC4 is negotiated. Because RC4 lacks modern salting and iteration mechanisms, attackers can extract this ticket from memory and brute-force the password offline at exceptionally high speeds.

Architectural Fix: Group Policy Enforcement

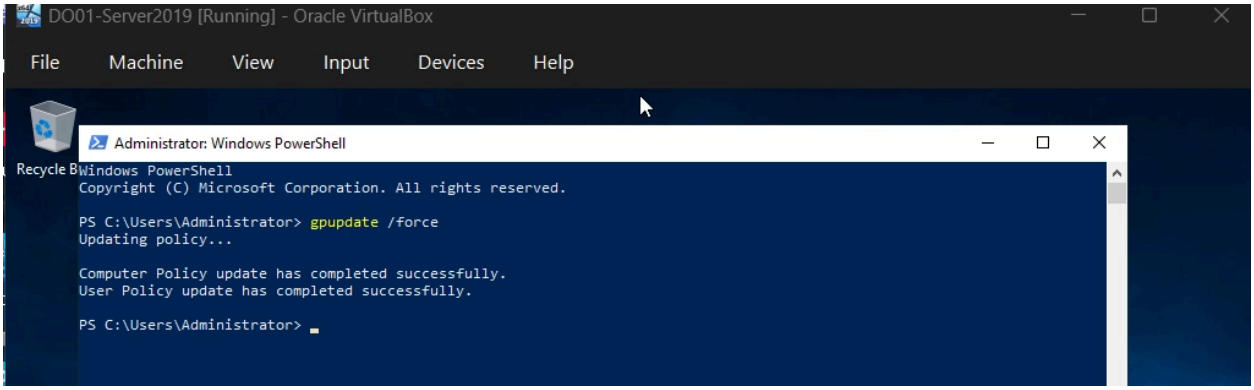
To neutralize weak ticket negotiation domain-wide, I modified the **Default Domain Policy** on the Domain Controller (`DC01`) to explicitly reject RC4 and mandate strong Advanced Encryption Standard (AES) algorithms.

- **Group Policy Object:** `Default Domain Policy` (Domain-level scope: `lab.local`)
- **GPO Configuration Path:** `Computer Configuration` ➔ `Policies` ➔ `Windows Settings` ➔ `Security Settings` ➔ `Local Policies` ➔ `Security Options`
- **Policy Setting:** `Network security: Configure encryption types allowed for Kerberos`
- **Enforced Configuration:**
  - ✅ `AES128_HMAC_SHA1`
  - ✅ `AES256_HMAC_SHA1`
  - ✅ `Future encryption types`
  - ❌ `DES_CBC_CRC` (Disabled)
  - ❌ `DES_CBC_MD5` (Disabled)
  - ❌ `RC4_HMAC_MD5` (Disabled)

To immediately apply the policy across the domain and flush existing policy caches, I executed a forced Group Policy update on both `DC01` and `Win11-Client`:

PowerShell

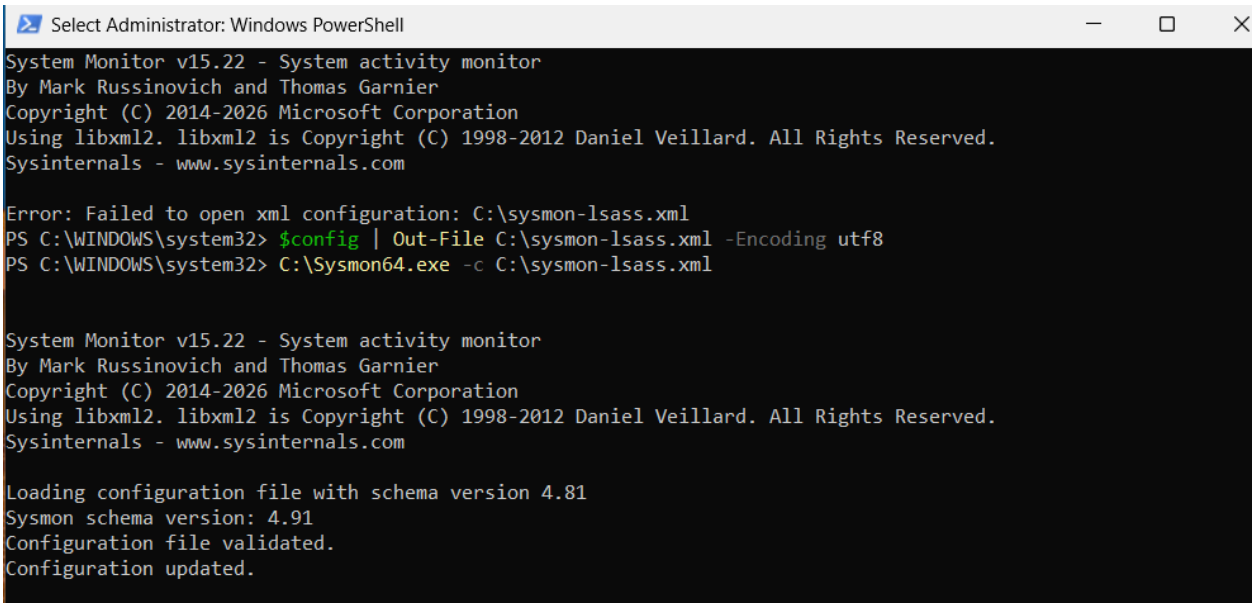
```
gpupdate /force
```



The "Why" (Architectural Impact)

By removing `RC4_HMAC_MD5` from the Kerberos Key Distribution Center (KDC) allowed encryption list on `DC01`, the KDC refuses to issue TGS tickets when an attacking tool explicitly requests encryption type `0x17`. Furthermore, forcing ticket negotiation up to `AES256_HMAC_SHA1` (`0x12`) introduces PBKDF2-like key derivation complexity, making offline brute-force cracking computationally infeasible when paired with strong service account passwords.

Remediation 2: Blocking LSASS Credential Access (LSA Protection & Defender Enforcement)



Technical Vulnerability

The Local Security Authority Subsystem Service (`lsass.exe`) caches active authentication tokens, Kerberos tickets, and NTLM hashes in system memory. In an unhardened Windows environment, a local administrator with the `SeDebugPrivilege` right can invoke native Windows APIs (such as `MiniDumpWriteDump` via `comsvcs.dll` or diagnostic utilities like `rdrleakdiag.exe`) to open a handle to `lsass.exe` and dump its memory contents to disk.

Architectural Fix: Layered Endpoint Hardening

First, I attempted to configure Local Security Authority (LSA) Protection (`RunAsPPL`) via the Windows Registry to run `lsass.exe` as a Protected Process Light (PPL):

PowerShell

```
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa" -Name "RunAsPPL" -Val
ue 1 -PropertyType DWord -Force
```

Lab Engineering Note: During validation, `RunAsPPL` did not prevent the `rdrleakdiag.exe` minidump inside the standard VirtualBox VM because full hardware-backed PPL enforcement requires UEFI/Secure Boot features not enabled on the virtual hardware profile.

To establish an active control within the virtualized architecture, I re-enabled Microsoft Defender Real-Time Protection and deployed the Attack Surface Reduction (ASR) rule designed to block credential theft from LSASS (`9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2`):

PowerShell

```
Add-MpPreference -ExclusionPath "C:\AtomicRedTeam"
Add-MpPreference -AttackSurfaceReductionRules_Ids 9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2 -A
ttackSurfaceReductionRules_Actions Enable
```

The "Why" (Architectural Impact)

Enabling Microsoft Defender alongside the LSASS ASR rule provides two layers of defense:

1. **AMSI (Antimalware Scan Interface):** Inspects PowerShell script blocks dynamically in memory before execution, catching known malicious test harnesses and dumping commands at the parser level.
2. **ASR Kernel Filtering:** The rule `9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2` monitors process handle requests targeting `lsass.exe` and blocks non-standard processes from obtaining memory-read access rights.

11. Phase 4: Purple Team Verification (Re-Testing)

To validate that the Phase 3 hardening controls neutralized the attack vectors, I re-executed the exact Atomic Red Team simulations from Phase 1 on `Win11-Client` and correlated both the terminal output and the Wazuh SIEM telemetry.

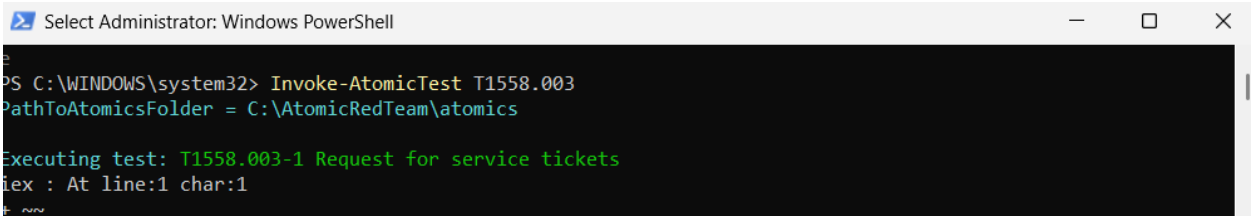
Re-Test 1: Kerberoasting Validation (T1558.003)

Execution

I purged the local Kerberos ticket cache and re-ran the Atomic Red Team Kerberoasting module (`Rubeus`):

PowerShell

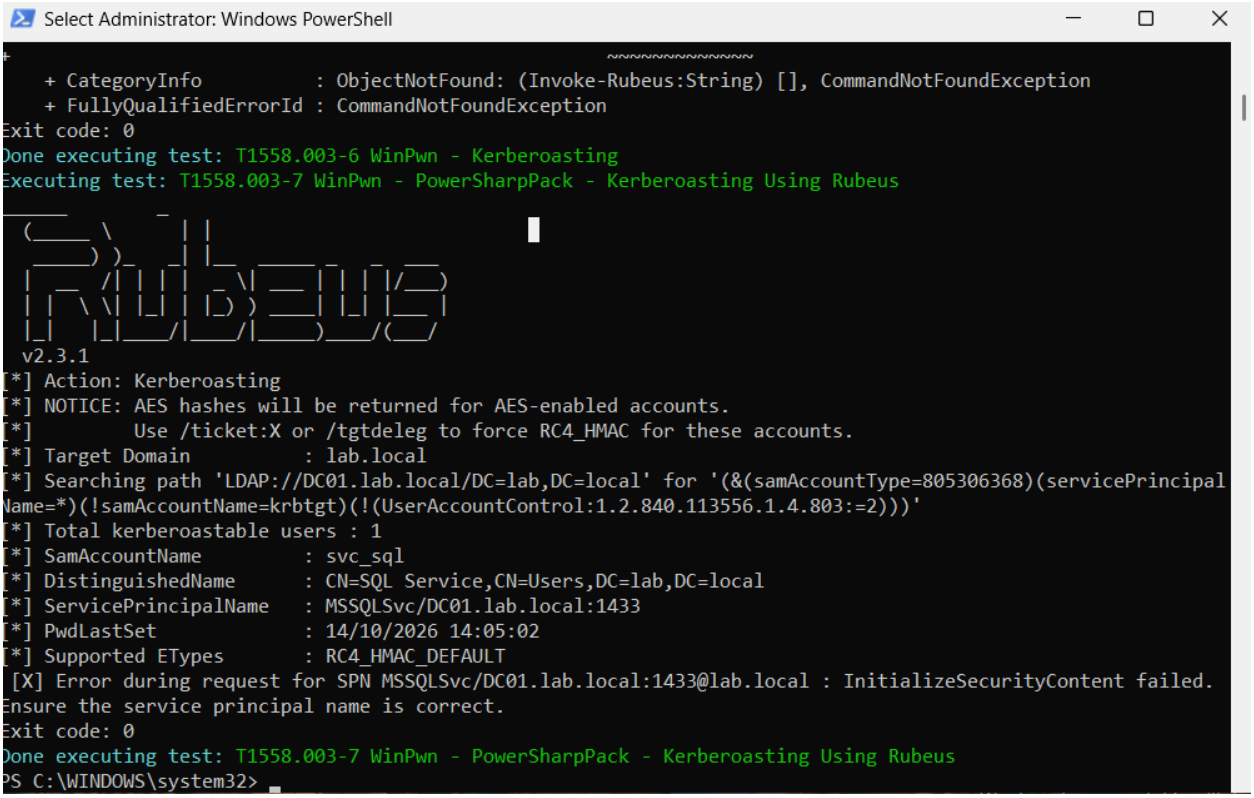
```
klist purge
Invoke-AtomicTest T1558.003
```



Verification & Correlation with Phase 1

- **Phase 1 Baseline (Vulnerable):** `Rubeus` successfully requested an RC4 ticket for `svc_sql`, extracted the `$krb5tgs$23$*svc_sql...` hash to the console, and generated Event ID `4769` (`TicketEncryptionType: 0x17`), triggering Wazuh Rule `100002` (Level 12).
- **Phase 4 Re-Test (Hardened):** `Rubeus` enumerated the `svc_sql` SPN (`MSSQLSvc/DC01.lab.local:1433`) via LDAP and saw that the account's attribute still listed `Supported ETypes: RC4_HMAC_DEFAULT`. However, when `Rubeus` attempted to request the RC4 TGS ticket from `DC01`, the Domain Controller's hardened Group Policy rejected the legacy encryption request.
- **Terminal Proof:** The attack failed with `[X] Error during request for SPN MSSQLSvc/DC01.lab.local:1433@lab.local : InitializeSecurityContext failed.` No `$krb5tgs$` hash was output.
- **SIEM Telemetry Proof:** Because no RC4 (`0x17`) ticket was issued by the KDC, Wazuh Rule `100002` generated **0 new alerts**, confirming the vulnerability was closed at the protocol negotiation layer.

Rubeus failing with `InitializeSecurityContext failed` after Group Policy disabled RC4 Kerberos ticket negotiation on DC01.



Re-Test 2: LSASS Credential Access Validation (T1003.001)

Execution

I re-executed the Atomic Red Team LSASS credential dumping test suite:

PowerShell

```
Invoke-AtomicTest T1003.001
```

Verification & Correlation with Phase 1

- **Phase 1 Baseline (Vulnerable):** The `T1003.001-13` sub-test (`rdrleakdiag.exe`) executed with `Exit code: 0`, writing `minidump_820.dmp` to the user's `Temp` directory. Concurrently, Sysmon logged Event ID `10` (`Process accessed` against `lsass.exe` with `GrantedAccess: 0x1000`), triggering 20 hits on Wazuh Rule `100003` (Level 12).

- **Phase 4 Re-Test (Hardened):** When `Invoke-AtomicTest` attempted to execute the `T1003.001-13` PowerShell harness invoking `rdrleakdiag.exe`, Microsoft Defender's Antimalware Scan Interface (AMSI) intercepted the script block prior to execution.
- **Terminal Proof:** Execution was terminated with `ParserError: This script contains malicious content and has been blocked by your antivirus software` (`FullyQualifiedErrorId: ScriptContainedMaliciousContent`, `Exit code: 1`). No minidump file was created.
- **SIEM Telemetry Proof:** Because the malicious script was blocked at the PowerShell parser stage before `rdrleakdiag.exe` could spawn or request a handle to `lsass.exe`, Sysmon recorded no new Event ID `10` entries for the test, and Wazuh Rule `100003` remained quiet.

Microsoft Defender / AMSI blocking the Atomic Red Team T1003.001-13 `rdrleakdiag.exe` LSASS dump execution (`Exit code: 1`).

